

CS261 Planning and Design Document

Young-Ha Chun Owen Green Archie Harrodine Tudor Irimies Yi Xiang Ng

Saturday 11th April, 2026

Contents

1	Introduction	2
2	Technical Description	2
2.1	System Architecture	2
2.2	Model Description	2
2.3	Metric Consideration	3
2.4	Database Design	3
3	Processes and Activities	4
3.1	User Interaction	4
3.2	Component Interaction	5
3.3	Risk Management and Modelling	5
4	UI Design	7
5	Testing	9
5.1	Unit Testing	9
5.2	Usability Testing	9
5.3	Integration Testing	10
6	Methodology	10
6.1	Plan	10
6.2	Risk Assessment	11
7	Sign Off	11

1 Introduction

This document outlines the design of a risk management web application. The following acronyms are used throughout: Application Programming Interface (**API**), User Interface (**UI**), Object Oriented Programming (**OOP**), Software Development Kit (**SDK**) and Amazon Web Services (**AWS**).

2 Technical Description

2.1 System Architecture

This was the first major design decision that was made, as it forms the basis behind all of the other major threads that we cover. The architecture chosen was the Model-View-Controller (MVC). This architecture is crucial to our product's functionality, as this design is built around providing users a multitude of ways that the system can be viewed and interacted with (CR1.1). By offering a separation of presentation, interaction and system data, MVC not only lets users effectively interact with their software projects in real time (CR1.2), but also enables us to divide the threads of work cleanly, between the model, the view and the controller.

The MVC architecture was chosen for the website design due to its division of labor and clear interaction structure, as shown in Figure 1, making it ideal for managing user interaction. In addition, the MVC architecture allows for a separation of concerns, making it easier to maintain and update the website in the future (DR4.2). In particular it means the implementation of adding in new metrics to track for the database, and how the website can handle it, can be tackled by separate teams.

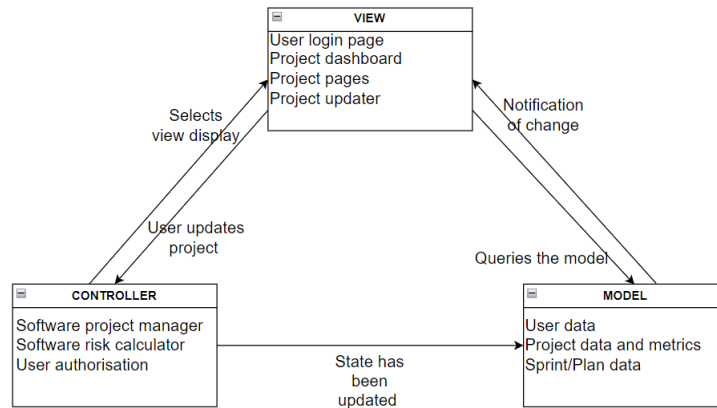


Figure 1: MVC diagram and detailing of features.

2.2 Model Description

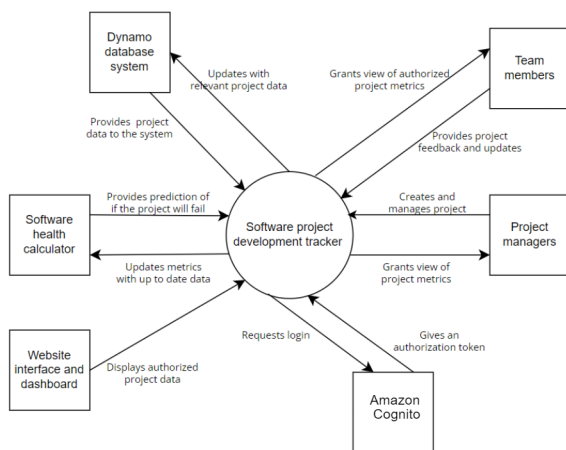


Figure 2: Context Diagram of the system

This context model captures the information about the key entities of the system. With the MVC architecture, it allows for a design which is not overly reliant on specialised OOP coding practices. Each component of the system is designed to interact with each other when necessary, and the inner workings of each component aren't so complex to require a large number of classes to be detailed. The more important aspect of this design to consider, is how we are to design each of these subsystems, and how they are to interact with each other. Therefore no class diagram is needed, as the information is encapsulated within the diagrams above.

With the underlying architecture being determined, this now allows for the actual model itself to be constructed. This takes the form of looking at the overall scope, with regards to a context model, and describing entity relationships. Firstly, the overall scope of the model is represented below in the following diagram. This breaks down the key aspects of what the system has to do, and the various systems that it makes use of to accomplish this.

2.3 Metric Consideration

The project's aim is the gathering and interpretation of various metrics used to track the progress of software. There are two ways this will be approached: firstly we will look at business practises ([CR2.2](#)), deriving how companies go about determining the current trajectory of a software project, and using that information to decide whether the software project should go ahead; secondly, we will take a look into the metrics that have been used for academic research in determining project success. The division is made to gain a wider perspective of the impact of metrics, and to come to our own final conclusions on our approach to designing the algorithm to find the risk of failure ([CR2.1](#)).

Additionally, we shall also be reaching out to various groups and companies to try to gain access to data sets that we can use to find patterns, and develop a more successful project risk calculator.

This section is now to cover the business research that was carried out. Amongst reading several fairly generic online articles detailing the reasons for why software engineering projects fail, the most information for this side was gleaned from an interview with a chief architect at a large defence company. This interview detailed the metrics which each sprint is evaluated upon.

- **Schedule:** This is rating if the requirements for a certain sprint have been met within the deadline set. The requirement and the associated resource is flagged upon review if not met.
- **Cost:** This is the rating determining if the project is within budget. This considers the total expenditure from resources, technologies, and realised risk.
- **Quality:** This is the rating of the quality and success of each completed requirement. This is ranked by either the developer upon submission, or by the project manager when evaluating the sprint.
- **Business change:** This is rating the impact of any internal or external restructurings of the company. This can take the form of new platforms, change in management, new office location etcetera.
- **Safety:** This regards how well a system can handle system failure or erroneous human actions. We take both how likely a fault is to occur, and how well the system tackles such a fault.
- **Resourcing:** This is the allocation of resources towards certain aspects and areas of a given project. Generally this evaluates the success of the current allocation.
- **Risk:** This tracks the amount of risk realised from the risk assessment. This may be considered with either a rating or using the monetary value of realised risk.
- **Stakeholder satisfaction:** This entails the happiness of the customer with the current build. We can expand upon this for certain requirements to be ranked by the customer.
- **Technical maturity:** This considers how far along a solution is to the problem. We look if there is a solution and a design. Additionally we take into account all realised and potential technical issues.
- **Sprint review:** This addresses the project manager's personal review of how the project is progressing.

All of these metrics are looked at for each sprint, and using that data we look at the trends presented for each metric. We use those trends to predict and anticipate how they may perform going forward, and what changes may need to be made to keep the project successful. For our project, including and tracking all of these would be severe scope creep, and so we must prioritise which we are to include and implement. The level of priorities are as such: schedule, cost, risk, resourcing, stakeholder satisfaction, sprint review, quality, technical maturity, business change ([DR2.2](#) , [CR4.2](#)). This ordering is derived from various articles, and what was discussed with the interviewer. In particular, remaining on schedule, remaining within budget, and ensuring enough redundancy to handle realised risk, were the three key areas which determine if a software project is likely to fail. Then technical maturity and business change were the two that weren't as important; business change as it is less common and often factored in; and technical maturity because a project is only undertaken after passing a feasibility study, which highly factors around these parts.

2.4 Database Design

For this project, we will be using AWS' DynamoDB as the database that stores all the necessary information needed, and Amazon Cognito to handle the logins. DynamoDB is one of Amazon's NoSQL Databases that works easily with AWS which we are using to host our project, and more importantly, it's free (for tiny scale projects). However, the service is scalable to meet increased demands. Additionally, DynamoDB is also compatible with Amazon Cognito and other programming languages through the use of AWS SDK ([DR1.2](#)).

Amazon Cognito will be the service we are using to manage our user's credentials. Whenever a new user account is created for a new employee / manager, the details are added into the user pool which then can be passed on to DynamoDB. Additionally, Amazon Cognito also provides other services such as the creation of administration accounts, updating user's passwords and migrating data from CSV files. One particular service provided by Cognito that we can utilize well in our software is the feature to send emails and SMS to users when an event is triggered. Such a feature allows us to update and remind users about upcoming deadlines and other such events. When a user logs in, their credentials are checked from a user pool, and upon successful authentication, the user is granted tokens which can then be exchanged for access to other AWS services (**DR1.3**).

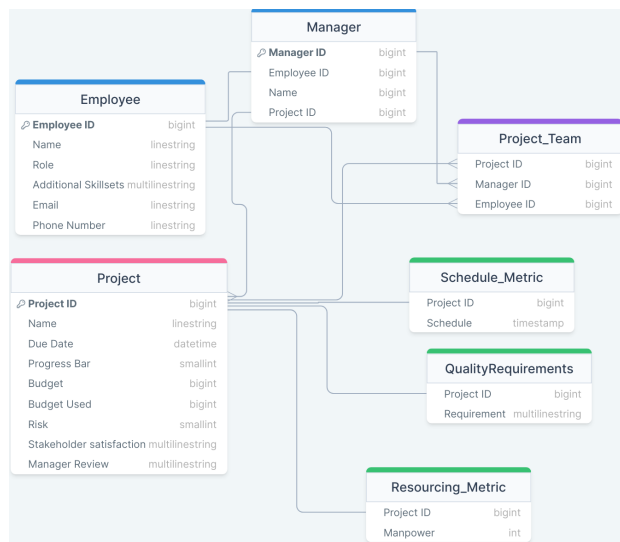


Figure 3: Database Entity Relationship Diagram

For our database, the use of data normalization by creating multiple tables would be optimal as it reduces data redundancy and duplication. The general idea would be to have a Project table, which would include all the key information related to the Project such as the name, ID, progress % and risk. This will easily be able to be expanded (**DR4.2**). Additional tables that hold information on other metrics used by the modelling part of our software will reference the project table. On top of that, we will have an Employee table and a Manager table to keep track of individual team members and their managers (**DR1.8**). Finally, the Projects and employees' information will be linked together by another table, 'Project Teams'. The database structure can be visualised using the figure above (Figure 3).

3 Processes and Activities

3.1 User Interaction

The user in this project has a variety of ways it needs to interact with the system. For product managers of teams, they will need to be able to create and edit software projects, be able to view all project details, and control the level of access that other team members possess (**CR1.6**). Other users will need to be able to view project details specified by the project managed, and be able to submit feedback and progress updates of the project.

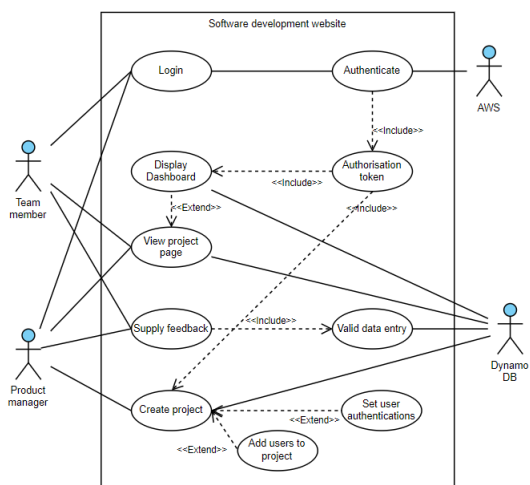


Figure 4 adeptly demonstrates the user's primary interactions, with a series of team members having access to the login page, the view of the projects (accessed from the dashboard) and giving feedback or updates to the project (**CR1.9**). The project manager has similar privileges, but the key difference is that they can also create projects, which will be indicated within the UI. The left-hand side has the two primary users of the team members and the project managers, whereas the right-hand side has the external third party tools we are making use of, as detailed earlier. The key things of note from this diagram is the extensions when creating a project, to be able to set different user restrictions on what metrics can be seen, and the option of adding people to the provided project (**CR1.6**).

Figure 4: Use-case diagram to showcase user interactions.

3.2 Component Interaction

The way that our main components interact is solidly reflected by our MVC architecture. There are a few areas that need further elaboration though. Firstly the login system must be touched upon due to its deeper ties with the external AWS system.

Figure 4 above demonstrates the first initial login process, accounting for team members and project managers. Additionally, it is tracked within the database if the user has used the website before, and if they haven't, a tutorial will be brought up (DR1.5), to help the user navigate the page for the first time. The key distinction here is our focus on checking the permissions of the user, since our focus is giving more control to the manager.

The next area of importance to tackle is in regards to the other major feature of user interaction within the system, and that is targeted toward the input of data, and the viewing of that data. The three systems of the user, the database and the risk recalculation program, all need to interact here to enable for a seamless update to the page, making it far more intuitive for the user to follow. The following sequence diagram shows this conceptually, of how the three components will interact with one and other in a given process (DR1.1).

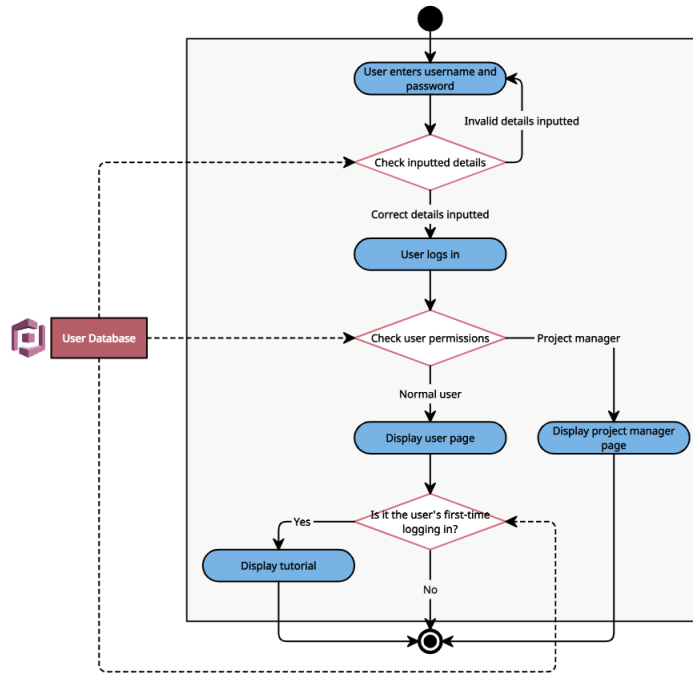


Figure 5: User login activity diagram.

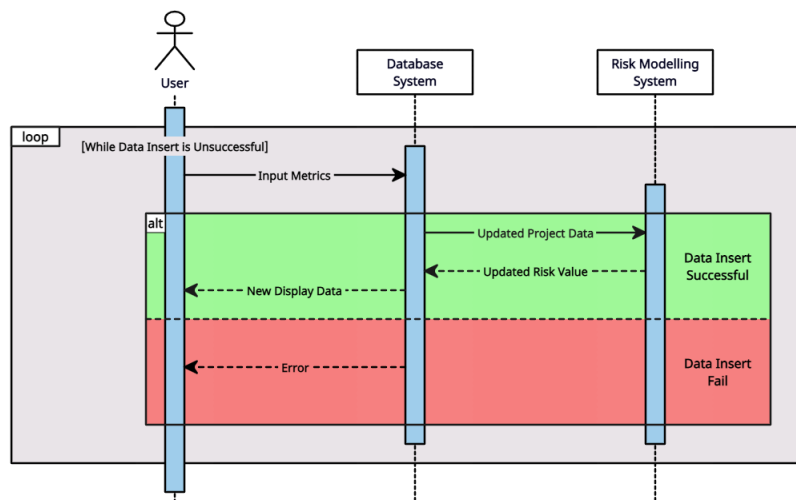


Figure 6: Sequence diagram of data entry.

3.3 Risk Management and Modelling

The study of risk management in software engineering has evolved with the discipline itself. As early as 1989 software risk management was being studied with it being acknowledged that software 'rework' costs already accounted for forty to fifty percent of total software development costs [2]. As time has evolved the complexities of the techniques used for risk management has increased with many Artificial Intelligence (AI) and Machine Learning (ML) techniques being applied to the problem. We consider several of these approaches before outlining our proposed modelling technique and detailing how it will be connected to the wider system.



Figure 7: The Átropos model

However, one of the main difficulties in applying models such as the one presented above to our problem is finding suitable datasets. A large number of suitable context histories are needed to build an accurate model and the attributes tracked by Filippetto et al. were not suitable for our use. Indeed, the need for suitable datasets has been identified in the literature with Shakut et al. proposing a dataset which would enable risk prediction in software projects [7]. They collected 299 requirements from open source software projects and tracked the probability, priority, impact, and fix cost amongst other attributes. This provides a more suitable basis to model the risk of projects in our system. The work of Shakut et al. was extended by Akumba et al. who applied a Naïve Bayes Classifier to the dataset described above [1]. They used seventy five percent of the data to train their model and tested it on the remaining twenty five achieving ninety eight percent accuracy. Risks were classified into five categories: Catastrophic, High, Moderate, Low, and Insignificant.

This classification of risk fits in with the model proposed by Chen and Deng who split risk severity into 5 levels enumerated A to E as displayed in Figure 8. These correspond to risk severity levels of Very little, Little, Medium, Serious, and Catastrophic. Deng and Chen used this matrix along with Deng entropy [4] in order to build a model of risk which accounted for the uncertainty in risk evaluation. However, they acknowledged that their model would need to be re-evaluated over time as software risk management is a dynamic process and would therefore need to dynamically weight attributes given their differing importance at different stages of a software project [3].

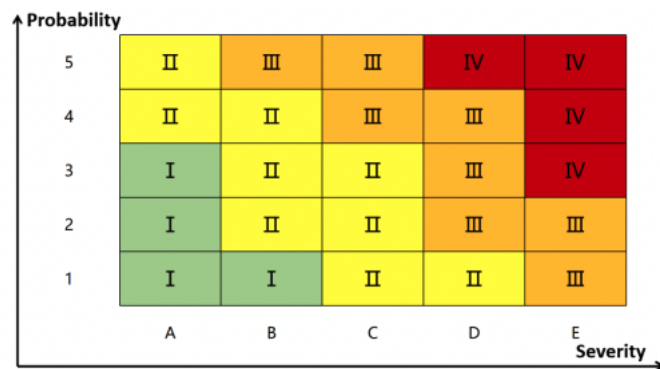


Figure 8: A Risk Evaluation Matrix [3]

We plan on using a Naïve Bayes Classifier in a similar style to Akumba in order to classify various risks affecting our project. We can use the dataset proposed by Shakut et al. in order to do this. This will allow us to effectively determine the severity of all of the risks affecting our project. We will then combine that with probabilistic information which will ideally be in the form of a user's estimate of how likely a given risk is to occur combined with risks from already existing projects. This will allow us to quantify risks into four levels as shown in Figure 8, which Chen and Deng denote as Low, Medium, Significant, and High [3].

One of the approaches we considered is presented in Figure 7 and was proposed by Filippetto et al. in 2020 [5]. It uses the idea of context histories which are collected throughout the project. When the risks of a new project are to be evaluated the system compares the characteristics and context histories of the new project with its existing database of projects and is then able to provide recommendations informed by similar previously completed projects. The model obtained eighty three percent accuracy when its performance was evaluated by experts on a dataset of 153 software projects from a financial company [5].

This approach will let us show the user which risk factors affecting their project are the most significant, which is crucial to making our data analysis easily understandable for the non technical user (DR2.4, CR2.1). The overall risk of a project can then determined by taking the weighted sum of these risk factors. This will likely require some experimental testing in order to fine tune the values so that the result is properly normalised between 0 and 1 (DR2.1).

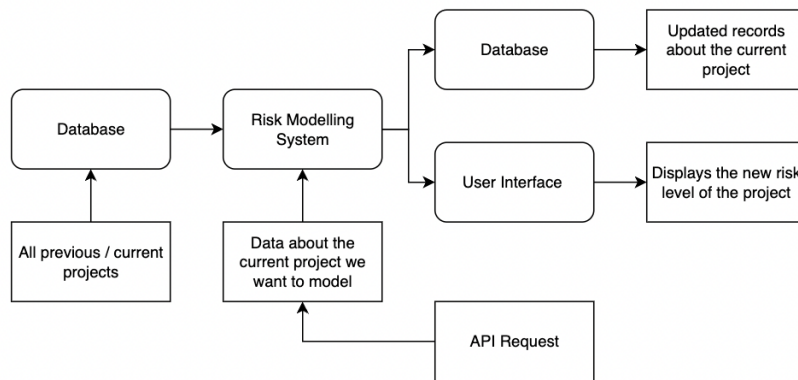


Figure 9: A Pipe and Filter diagram showing how the risk modelling system interacts with other system components.

4 UI Design

The website aims to provide an all-in-one platform for tracking and managing software projects. The platform will be centred around a dashboard, which allows users to see the software projects they manage and receive real-time updates on their progress. Project members will have the ability to view a subset of information about the project, as dictated by the project manager (CR1.2), and they will be able to access detailed information about each project by selecting it from the dashboard. The platform will track a variety of metrics, including project progress, and the amount of information displayed will depend on the user's current permission setting. To ensure an easy and intuitive experience, the platform will also provide users with a simple and straightforward method for updating the details of the project as it progresses, allowing for seamless management and tracking of each sprint or milestone. With the help of a navigation bar on the left-hand side, the user will be able to access the main pages of the website, as well as their list of co-workers, with a possible representation of a green circle adjacent to coworkers that are currently active on the website. Furthermore, the navigation bar provides access to a settings button and profile page, providing additional customization options for users to personalise their experience, and change their preferences and update their profile information, further tailoring the website to their specific needs (CR1.8). The most important pages accessed through the navigation bar are the following:

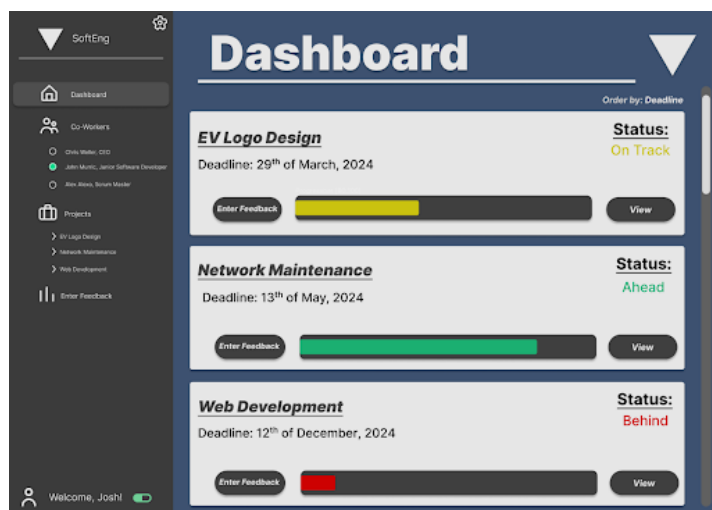


Figure 10: The UI Dashboard

Dashboard (Figure 10): This page will provide an overview of all software projects being tracked, displayed in a clear and organised manner. Each project will be represented by a card that shows a small amount of detail such as project name, status, and progress. The user will be able to view all projects at a glance and quickly navigate to a specific project page for more information. The group considered two different methods for accessing the Project Page: by clicking on the card corresponding to the project, or by clicking on a button leading toward the Project Page. Ultimately, the latter has been chosen, due to its intuitiveness and minimal training required (CR1.9), as a button is the general method used to access information on most websites.

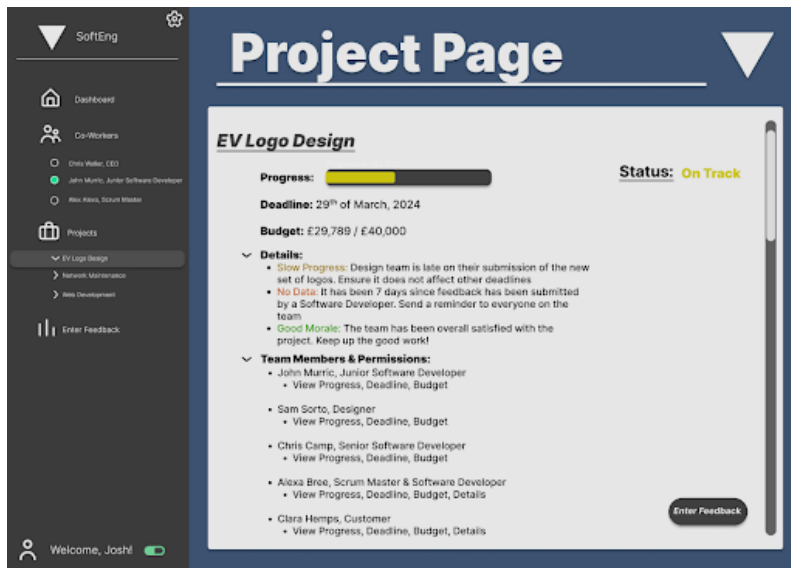


Figure 11: The UI Project Page

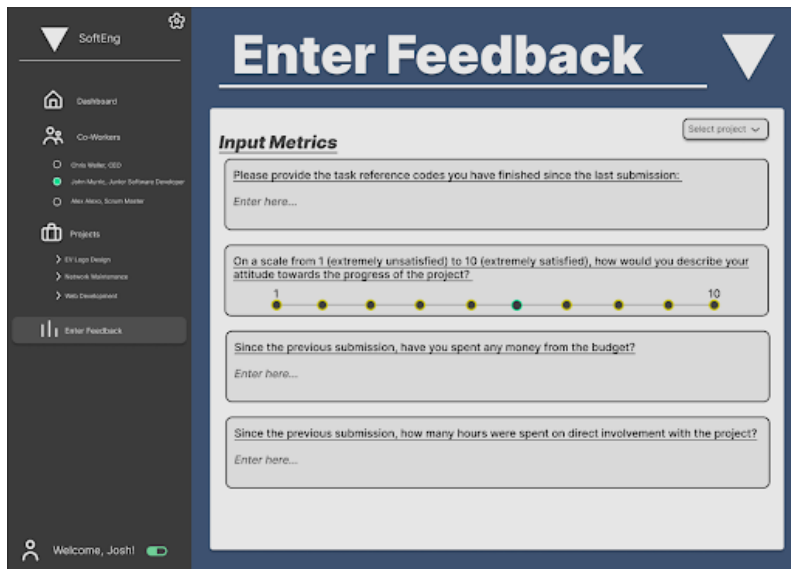


Figure 12: The UI Feedback Page

Additionally, the project manager will have access to a button for adding a new project to the system located at the bottom of the page. This will display an overlay which will require the project name to be inputted along with the initial metrics. The scroll bar has been added to provide the user with the ability to scroll through projects instead of viewing all at once, removing clutter from the screen and keeping a minimalist aesthetic. Lastly, an “Order By” button has been introduced, to allow users to more easily navigate projects, by sorting through deadlines, statuses, or other key features, thus providing more customisability.

Project Page (Figure 11): This page will provide more in-depth information about a specific software project. The user will be able to view details such as project description, tasks, milestones, team members, and progress, depending on the permissions given (more about permissions will be discussed later). Furthermore, a button will be available to the project manager which will present an overlay where members can be added to a project or removed from one via a worker list. The information will be presented in a clear and organised manner, allowing the user to easily track the progress of the project. Throughout this page and the dashboard, colour has been used to emphasise critical aspects of the project, such as its progress and possible improvements. However, to ensure better accessibility, the same information provided by the colours can be understood without its aid, for example the progress bar changing colour depending on the project’s status, however the length of the bar may provide the same context.

Feedback Page (Figure 12): This page will provide a platform for users to submit feedback about the software projects. The user will be able to submit their feedback in a form, selecting which project the feedback applies to, and may include some of the following criteria: a list of tasks accomplished, morale and attitude towards the project, budget update, number of requirements completed, and a number of code-related criteria, such as bugs fixed and code coverage percentage. The feedback will be processed, updating the database and later displayed on the project page, thus being used to improve the overall performance of the software project. Overall, the feedback form is made to be intuitive and quick, thus incentivising the workers to update the progress often without hindering their ability to work.

The website aims to provide a high degree of customisability for users to submit feedback. With three main methods available - through the dashboard for each individual project, through the project page, and through the dedicated feedback page - users are empowered to choose the method that works best for them, creating a tailored experience. This approach to feedback submission reinforces the website’s commitment to user-centred design and provides a user-friendly interface.

The design of the buttons on the website follows the principles of Gestalt psychology, with a consistent style and convenient placement to reduce cognitive load on users. The similarity principle ensures a cohesive look for buttons, establishing a clear

visual hierarchy and making it easier for users to understand the relationships between different elements on the page. This design approach minimises the burden on users' long term memory and instead focuses on short term memory and reasoning, creating a streamlined and efficient experience ([CR3.2](#)).

In addition to its clear and intuitive design, the web page will also be fully responsive and adjustable to different devices. This means that the page will automatically adapt to the screen size of the device being used, providing a seamless user experience regardless of whether the user is accessing the site on a desktop computer, tablet, or smartphone. This will make it easy for users to track the progress of software projects and provide feedback no matter where they are, ensuring that they are always up to date with the latest information. The page is also designed with accessibility in mind. This includes features such as appropriate colour contrast, clear and descriptive headings, and logical page structure, which will make it easier for users with disabilities to navigate the site and access the information they need. By making the website accessible, we are committed to providing a positive user experience for all users, regardless of their ability. Finally, it is important to note that the upside down triangle used on the website and the name "SoftEng" are placeholder elements, meant to be replaced with the company's own logo and branding. The triangle serves as a symbol for the company's vision and mission, while the name "SoftEng" is a temporary moniker meant to represent the company until a final name is chosen. The company is encouraged to personalise the website with their own logo and branding, as this will help to create a unique and memorable identity that sets the company apart from its competitors.

A tutorial will be created for first-time visitors of the website to help familiarise them with the features of the dashboard, project page, and feedback submission methods, by guiding users through the user interface, highlighting key features, making it easier for them to navigate the site and utilise all the tools available to them. The first-time user will be identified by checking a status attribute which will be stored in the user database. Depending on this status, the user will either proceed to their personal dashboard or be redirected to the tutorial ([DR1.5](#) , [CR1.5](#)).

5 Testing

The way our testing is designed, is such that each sprint we ensure that our project, and our risk model stays on track. Within larger software projects, it is so easy to divert from the original requirements; hence each week during the sprint evaluation, the scrum master will be tracking not only if the stories meet their goals, but shall also be evaluating how focused the result is when compared to the task set.

5.1 Unit Testing

Most of the unit testing will be trivial ([DR3.1](#)), and is not worth writing about; especially in regards to the website testing as that is an active testing process. However the area that is worth focusing on is the testing of the risk evaluator. We will need a consistent and effective method to ensure that our algorithm (whatever its final form) will provide an accurate interpretation of the project's health. This will be achieved as such. Within the first sprints, the controller thread will be tasked with additionally writing a series of *dummy* projects, which are a series of projects that have obvious and notable attributes which let us determine ourselves its health. For example, there will be *dummies* that are significantly over budget, behind schedule, poorly allocated resources etcetera. Using their own judgement, the projects will be given a risk value and error bounds associated with it. From there, there will be a series of automated tests each sprint which will evaluate these dummy projects, and the value that it attains will be tested to see how close it falls to the *actual* value. Furthermore, this may be built upon using real-world data, those being added to these automated tests ([DR3.3](#)). This method is particularly useful, for if there are a few models to test against, and the best performing can be selected from the iteration. However, this approach does run the risk of human error, with us poorly deciding values, but with these hard to quantify metrics, this feels to be the best approach.

When it comes to testing our database, Amazon SDK once again is used to ensure that DynamoDB is working to our expectations. Using the SDK, we can test a local instance of the database by performing inserts, queries, inserts/deletions and updates on mock datasets and making sure that the result is what we are expecting. Additionally, we could use mock libraries with known legal states, and ensure that each database operation will not violate any constraints or result in an inconsistent state ([DR1.2](#) , [DR3.2](#)).

5.2 Usability Testing

The usability testing process is aimed at checking how accessible the interface is to a user and their experience using it ([CR3.3](#) , [DR3.2](#)). By analysing the user during the test and asking for their thought process and feedback, we will be able to learn

about user preferences and also identify problems with the system that can be improved. Solutions to these issues could then be worked on during the next sprint cycle for the upcoming iterations of the software, to enhance the user experience.

We will be conducting 2 rounds of in-person testing sessions on 5 participants. The first round of tests will be aimed at getting a general understanding of the user's experience with the interface, while the second round will be held to iron out any small aspects that may need to be changed. Due to the circumstances we will be unable to recruit testers from Deutsche Bank and therefore have decided that a suitable compromise would be Computer Science students unrelated to our project. For each test we will have a set of tasks that the participant will follow, with the test being recorded to help with validating the test results and future analysis. The list of tasks will be as follows:

- Log in.
- Go to the dashboard.
- Create a project.
- Add and allocate members of the project.
- View the project page.
- Go to the feedback page and provide feedback for a sprint.

One of the project members will provide the participant with a list of tasks, while also keeping track of a number of metrics to help evaluate the testing process.

The metrics to be recorded will be split into two categories:

- Quantitative: Time taken for each task, task completion (success/failure), number of times the user struggled/encountered an issue.
- Qualitative: User satisfaction feedback received after the test, ease of use, participant stress levels and user's verbal and non-verbal feelings throughout the test.

The guide will encourage the participant to provide constant feedback and voice their thoughts, actions and feelings as much as possible, to maximise the transparency of the user's reasoning process. Problems that arise during the testing will be listed and then later rearranged in order of severity at the end of the process. In this way, we will ensure that the most critical issues will have priority during development.

Once all the usability tests have been completed, all information (test metrics, notes made during test, feedback) will be collated and used to create a set of improvements that could be made to the system.

5.3 Integration Testing

When integration testing, we must ensure that all of the components of the system work together coherently. For this project, what this entails is that the user is able to log on successfully, view their dashboard, create and access a project page and update said project. This requires that the information displayed is accurate and correct to what is held in the database. These steps, are near identical to what is used for our usability testing. Hence for our integration tests, our developers will perform the same exercises set for usability testing, only here we will also be ensuring the correct display of data from what should be in the database. Furthermore, we will also check that input validation is behaving as expected and that any errors are handled without breaking the system (CR3.2).

6 Methodology

6.1 Plan

Due to the division of labour present from the MVC architecture, in combination with our scrum methodology, the plan provided within the requirements document does not need much further elaboration. There are three main threads of work, the database (model), the website (view) and the risk evaluation (controller). At the start of each sprint, the team's resources are allocated to each thread, with a set of requirements found through discussions between the project manager and the scrum master (further detailed in requirement document). Then at the end of each sprint, each thread is evaluated for its development, for defining new requirements. As for handling the integration of the three threads, as each thread gets closer to completion, each sprint evaluation will serve for discussions on the integration between the elements. The reason why this methodology of scrum was chosen, was because it let us be able to continually and effectively build on our project. As we are all students, who have occupied lives, and limited time, the planning ahead that we can do is limited. When weighing up the priorities had for this project, it was important to us that we can easily adapt its trajectory for when our busy university lives enable it. This design document

is planned such that it can be expanded upon with new metrics, new modelling techniques, and new features without escalating into unachievable scope creep. Scrum does offer disadvantages, especially with regards to our lack of a real customer, yet this actually turns into a strength when we use our own research, and testing, to determine our requirements for each following sprint (DR4.3).

6.2 Risk Assessment

When considering the plan ahead, if the research for what makes projects has told us anything, ensuring risk is appropriately accounted for is essential for ensuring success. Below, are the most important risks that have had to be considered, so in the event they happen, our project's development is not derailed.

Risk: A description of the potential risk.

Likelihood: A scale from 1 to 5, judging how likely the risk is to occur, where 1 is near impossible, and 5 is incredibly likely.

Severity: A scale from A to E, judging how severe the impact would be if the risk is realised, where A is not very damaging to the project, and E is very dangerous to the health of the project.

Risk Register: This is the rating determined by the evaluation using the risk register. [6]

Control: Potential solutions to the proposed risk, and its potential to impact the project.

Risk	Likelihood	Severity	Risk Rating	Control
Deadlines not being met. This regards both internally set deadlines, and deadlines set by the university.	4	D	Extreme.	Constant checkups with members of the team to gauge their progress. Reallocate resources if members are not well suited to a particular task, or are falling behind. Prioritization of the necessary requirements to be completed first. Reworking the next sprint to accommodate areas that we are falling behind on.
Lack of necessary technical skills to complete a task. For example not having experience with a certain programming language required for a specific area of the project.	3	B	Moderate.	Ensure that in the planning of a sprint, we allocate enough time for also learning the technical skills needed. On our regular meetings, check how training is going, and determine if more time or resources need to be allocated to that area.
Team member personal circumstances. Due to whatever reason, a member of the team being unable to complete their tasks for the project, as a result of personal circumstance.	2	D	High	Reallocation of tasks amongst other team members to distribute out the work to be completed. Prioritization of requirements, so that in the event we need to cut back, the transition is as swift and easy as possible. Regular team meetings, and collaborative sessions, so in the event someone is unable to work, their specialties aren't entirely limited to one person. Regular personal checkups, to ensure team members are mentally and physically alright; as well as using them to gauge if they may be unable to complete the tasks of the current sprint.
Amazon web services have a legal or policy change. The external factor of AWS means unexpected events may occur.	1	D	Moderate	Have the team do light research into a potential alternative way to host the user login and database. Alternatively, use existing systems in place from team members web-development module coursework.
Sensitive information leak. If our website leaks sensitive or personal information.	1	E	High	Before each prototype, ensure that the website is at no risk of revealing or releasing information that those without authentication are not privy to.
White box testing failure. This is where an external tester finds the interface to be awfully designed.	3	A	Moderate	Hold collaborative design sessions where two or more people work on the interface, so each can keep the other in check, and reduce the number of potentially poor design decisions. In the event of an extremely poor reception to the interface, allocate more resources to the story for the following sprint, to prototype more interface options.
Unable to gain large enough datasets for modelling our project health evaluator.	3	B	Moderate	Ensure that we do not have a reliance on the need for a machine learning approach. Encourage research into probabilistic methods. In early stages of development reach out to a large number of academic sources, and businesses.
Scope creep. The sprint requirements escalating to an unachievable amount.	3	D	High	Often discussions within the group to ensure that new requirements are feasible. Whenever the requirements are expanded upon for each sprint, take goals that we want, and always reduce them in some way (complexity/amount).

7 Sign Off

The source code for the project along with suitable documentation and a final report will be delivered by Monday 13th March.

References

- [1] Beatrice Akumba, Samera Otor, Iorshase Agaji, and Barnabas Akumba. A predictive risk model for software projects' requirement gathering phase. *International Journal of Innovative Science and Research Technology*, 5:231–236, 2020.
- [2] Barry Boehm. Software risk management. In C. Ghezzi and J. A. McDermid, editors, *ESEC '89*, pages 1–19, Berlin, Heidelberg, 1989. Springer Berlin Heidelberg.
- [3] Xingyuan Chen and Yong Deng. An evidential software risk evaluation model. *Mathematics*, 10:2325, 2022.
- [4] Yong Deng. Deng entropy. *Chaos, Solitons Fractals*, 91:549–553, 2016.
- [5] Aleksandro Souza Filippetto, Robson Lima, and Jorge Luis Victória Barbosa. A risk prediction model for software project management based on similarity analysis of context histories. *Information and Software Technology*, 131:106497, 2021.
- [6] monday.com. What is a risk register?, 2022. <https://monday.com/blog/project-management/risk-register/> Last Accessed 5th February 2023.
- [7] Zain Shaukat Shaukat, Rashid Naseem, and Muhammad Zubair. A dataset for software requirements risk prediction. In *2018 IEEE International Conference on Computational Science and Engineering (CSE)*, pages 112–118, 2018.